PROJECT REPORT

Movie Recommendation System

Name- **Alankrit Sajwan**

Reg.No- **25BAI10512**

1. Introduction

1.1 Background

Recommendation systems have become a fundamental part of modern digital platforms. From e-commerce to video streaming, these systems help users discover content that aligns with their preferences, saving time and improving user experience. With an ever-growing volume of movies being produced and catalogued every year, the need for intelligent, automated recommendation tools is higher than ever.

This project presents a content-based movie recommendation system built entirely in Python, trained on the TMDB 5000 Movies dataset, and deployed as an interactive web application using the Streamlit framework. The system analyses multiple content features of each movie — including plot overview, genres, keywords, cast, and director — and uses natural language processing (NLP) along with cosine similarity to recommend five movies most similar to any given selection.

1.2 Motivation

The primary motivation for this project was to build an end-to-end machine learning pipeline: from raw data ingestion and preprocessing, through feature engineering and model training, to deployment as a functional web interface with real movie poster fetching via the TMDB API. This demonstrates practical skills in data science, NLP, and Python web application development.

1.3 Objectives

- Build a content-based filtering recommender using the TMDB 5000 dataset.
- Preprocess and engineer features from structured JSON-like columns (genres, keywords, cast, crew).
- Apply NLP techniques including tokenization, stemming, and vectorization.
- Compute a cosine similarity matrix over 1,494 movies.
- Deploy an interactive web app on localhost using Streamlit with live movie poster fetching.

2. System Overview

The system is divided into two major components: (1) a Jupyter Notebook-based training pipeline (movie-recommender.ipynb) that preprocesses the data, builds feature vectors, and computes similarity scores; and (2) a Streamlit web application (app.py) that loads the pre-computed results via Pickle files and exposes them through a browser-based interface.

Component	File	Description
Training Pipeline	movie-recommender.ipynb	Data loading, preprocessing, vectorization, similarity computation, Pickle export
Web Application	app.py	Streamlit UI, Pickle loading, TMDB API poster fetching, recommendation display
Serialized Model	movie.pkl / similarity.pkl	Pre-computed DataFrame and 1494x1494 cosine similarity matrix for instant lookup

Component	File	Description
Dataset	tmdb_5000_movies.csv tmdb_5000_credits.csv	Raw TMDB data containing movie metadata, credits, genres, keywords, cast and crew

3. Dataset

3.1 Source

The project uses the TMDB 5000 Movie Dataset, which contains metadata for approximately 5,000 movies sourced from The Movie Database (TMDB). The dataset is split across two CSV files:

- tmdb_5000_movies.csv — Contains fields including budget, genres, homepage, id, keywords, original_language, original_title, overview, popularity, production companies, release_date, revenue, runtime, spoken_languages, status, tagline, title, vote_average, and vote_count.
- tmdb_5000_credits.csv — Contains movie_id, title, cast (JSON array), and crew (JSON array).

3.2 Dataset Characteristics

After merging and cleaning, the working dataset used for model training had the following properties:

Property	Value
Raw rows (before cleaning)	4,809
Rows after dropna()	4,806 (3 rows with missing overview dropped)
Rows in final model	1,494 (unique movies with full feature set after merge)
Total columns (merged)	23
Features used for recommendation	overview, genres, keywords, cast (top 3), crew (director only)
Missing values (after cleaning)	0 across all used columns

4. Data Preprocessing Pipeline

A robust preprocessing pipeline was implemented in the Jupyter Notebook to transform raw CSV data into a clean, vectorisable feature representation. The pipeline consists of six sequential stages.

Step 1 — Data Loading & Merging

Both CSV files were loaded using pandas and merged on the title column using an inner join. This produced a combined DataFrame of 4,809 rows and 23 columns. Redundant columns were dropped to retain only the seven columns relevant to content-based recommendation: movie_id, title, overview, genres, keywords, cast, and crew.

```
movie = pd.merge(movies, credits, on='title')
movie = movie[['movie_id','title','overview','genres','keywords','cast','crew']]
```

Step 2 — Null Value Handling

A null-value audit was performed using movie.isnull().sum(). Three rows contained null values in the overview column. These were removed using dropna(), after which all relevant columns contained zero null values. The runtime column had 2 missing values and tagline had 844, but these columns were not used in the model and were therefore not imputed.

```
movie.dropna(inplace=True)
```

Step 3 — JSON Column Parsing

Four columns (genres, keywords, cast, crew) stored their values as stringified Python list-of-dict structures rather than actual Python objects. These were parsed using Python's ast.literal_eval() function. Two separate converter functions were written:

- convert(obj) — Extracts all name values from a parsed list. Used for genres and keywords.
- convert1(obj) — Extracts only the first three name values. Used for cast to limit to top-billed actors.
- fetch_director(obj) — Iterates through crew entries and returns the name of the person whose job field equals 'Director'.

```
def convert(obj):
    return [i['name'] for i in ast.literal_eval(obj)]


def convert1(obj):                      # top 3 cast only
    return [i['name'] for i in ast.literal_eval(obj)[:3]]


def fetch_director(obj):
    return [i['name'] for i in ast.literal_eval(obj) if i['job'] == 'Director']
```

Step 4 — Text Normalisation

Multi-word feature values such as 'Science Fiction' or 'Sam Worthington' were collapsed by removing internal spaces. This ensures that CountVectorizer treats them as single atomic tokens rather than two unrelated words, preventing false co-occurrence matches.

```
movie['genres']   = movie['genres'].apply(lambda x: [i.replace(' ','') for i in
x])
movie['keywords'] = movie['keywords'].apply(lambda x: [i.replace(' ','') for i in
x])
movie['cast']     = movie['cast'].apply(lambda x: [i.replace(' ','') for i in x])
movie['crew']     = movie['crew'].apply(lambda x: [i.replace(' ','') for i in x])
```

Step 5 — Overview Tokenisation

The plot overview was split into individual word tokens using Python's str.split() method. This converts the free-text overview string into a list of words, consistent with the list format of the other feature columns.

```
movie['overview'] = movie['overview'].apply(lambda x: x.split())
```

Step 6 — Tags Column Construction

All five feature lists (overview, genres, keywords, cast, crew) were concatenated into a single list column called tags. This unified column was then joined into a single lowercase space-separated string per movie, forming the input corpus for vectorisation.

```
movie['tags'] = movie['overview'] + movie['genres'] + movie['keywords'] +
movie['cast'] + movie['crew']
new_df = movie[['movie_id', 'title', 'tags']]
new_df['tags'] = new_df['tags'].apply(lambda x: ' '.join(x).lower())
```

5. NLP Processing & Model Training

5.1 Stemming

Before vectorisation, all words in the tags column were stemmed using the Porter Stemmer algorithm from the NLTK library. Stemming reduces inflected words to their root form (e.g., 'loved' → 'love', 'acting' → 'act'), which reduces vocabulary size and improves similarity matching across semantically related words.

```
from nltk.stem.porter import PorterStemmer
ps = PorterStemmer()


def stem(text):
    return ' '.join([ps.stem(word) for word in text.split()])


new_df['tags'] = new_df['tags'].apply(stem)
```

5.2 Feature Vectorisation

The processed tags column was transformed into a numerical feature matrix using sklearn's CountVectorizer. The vectoriser was configured with a vocabulary limit of 5,000 features and English stop word removal. This produced a sparse matrix of shape (1494, 5000), where each row represents a movie and each column represents the frequency of a word in the vocabulary.

```
from sklearn.feature_extraction.text import CountVectorizer
cv = CountVectorizer(max_features=5000, stop_words='english')
vectors = cv.fit_transform(new_df['tags']).toarray()
# Output shape: (1494, 5000)
```

5.3 Cosine Similarity Computation

Pairwise cosine similarity was computed across all 1,494 movie vectors using sklearn's cosine_similarity function. The result is a symmetric (1494 × 1494) similarity matrix where each entry [i][j] represents the cosine similarity between movie i and movie j, ranging from 0.0 (no similarity) to 1.0 (identical).

```
from sklearn.metrics.pairwise import cosine_similarity
similarity = cosine_similarity(vectors)
# Output shape: (1494, 1494)
```

Cosine similarity was chosen over Euclidean distance because it measures the angle between two vectors rather than the absolute distance, making it more robust to differences in document length.

Two movies with similar thematic content will have a high cosine similarity even if their tag lists differ in length.

5.4 Recommendation Function

A recommend() function was implemented that, given a movie title, fetches its row index in the similarity matrix, sorts all other movies by their similarity score in descending order, and returns the titles of the top five most similar movies (excluding the input movie itself).

```
def recommend(movie):
    movie_index = new_df[new_df['title'] == movie].index[0]
    distances   = similarity[movie_index]
    movie_list  = sorted(list(enumerate(distances)), reverse=True, key=lambda x:
x[1])[1:6]
    for i in movie_list:
        print(new_df.iloc[i[0]].title)
```

5.5 Sample Output

The following output was generated during testing in the notebook:

```
recommend('Batman Begins')


Kung Fu Panda
Kung Fu Panda 2
The Peanuts Movie
Big Trouble in Little China
Khumba
```

5.6 Model Serialisation

Both the cleaned movie DataFrame (new_df) and the similarity matrix were serialised to disk using Python's Pickle library, allowing the Streamlit application to load pre-computed results at startup without re-running the training pipeline.

```
import pickle
pickle.dump(new_df,      open('movie.pkl',      'wb'))
pickle.dump(similarity,  open('similarity.pkl', 'wb'))
```

6. Streamlit Web Application

6.1 Overview

The web application (app.py) is built with Streamlit, a Python framework for rapidly building data-driven web applications. The app runs locally on http://localhost:8501 when launched via the command streamlit run app.py. It provides a simple, clean interface for users to select a movie and receive five personalised recommendations, each displayed with its official poster image.

6.2 Application Architecture

6.2.1 TMDB Poster Fetching

A fetch_poster() function makes an HTTP GET request to the TMDB API endpoint, passing the movie_id to retrieve the poster_path, which is then combined with the TMDB image base URL to form the full poster URL.

```
def fetch_poster(movie_id):
    url  = 'https://api.themoviedb.org/3/movie/{}?api_key=<KEY>'.format(movie_id)
    data = requests.get(url).json()
    return 'https://image.tmdb.org/t/p/w500/' + data['poster_path']
```

6.2.2 Recommender Function

The recommender() function is an adapted version of the notebook's recommend() function. Instead of printing results, it returns two parallel lists: recommended_movie_names (titles) and recommended_movie_posters (poster URLs). The similarity matrix is indexed using the movie's position in the DataFrame.

```
def recommender(movie_name):
    index     = movie[movie['title'] == movie_name].index[0]
    distances = sorted(list(enumerate(similarity[index])), reverse=True,
key=lambda x: x[1])
    names, posters = [], []
    for i in distances[1:6]:
        movie_id = movie.iloc[i[0]].movie_id
        posters.append(fetch_poster(movie_id))
        names.append(movie.iloc[i[0]].title)
    return names, posters
```

6.2.3 UI Components

The Streamlit UI is composed of the following components:

- st.selectbox() — Dropdown populated with all 1,494 movie titles from the loaded DataFrame.
- st.button('Show Recommendation') — Triggers the recommendation pipeline on click.
- st.columns(5) — Creates a five-column layout.
- st.text() / st.image() — Renders each recommended movie's title and poster image within its column.

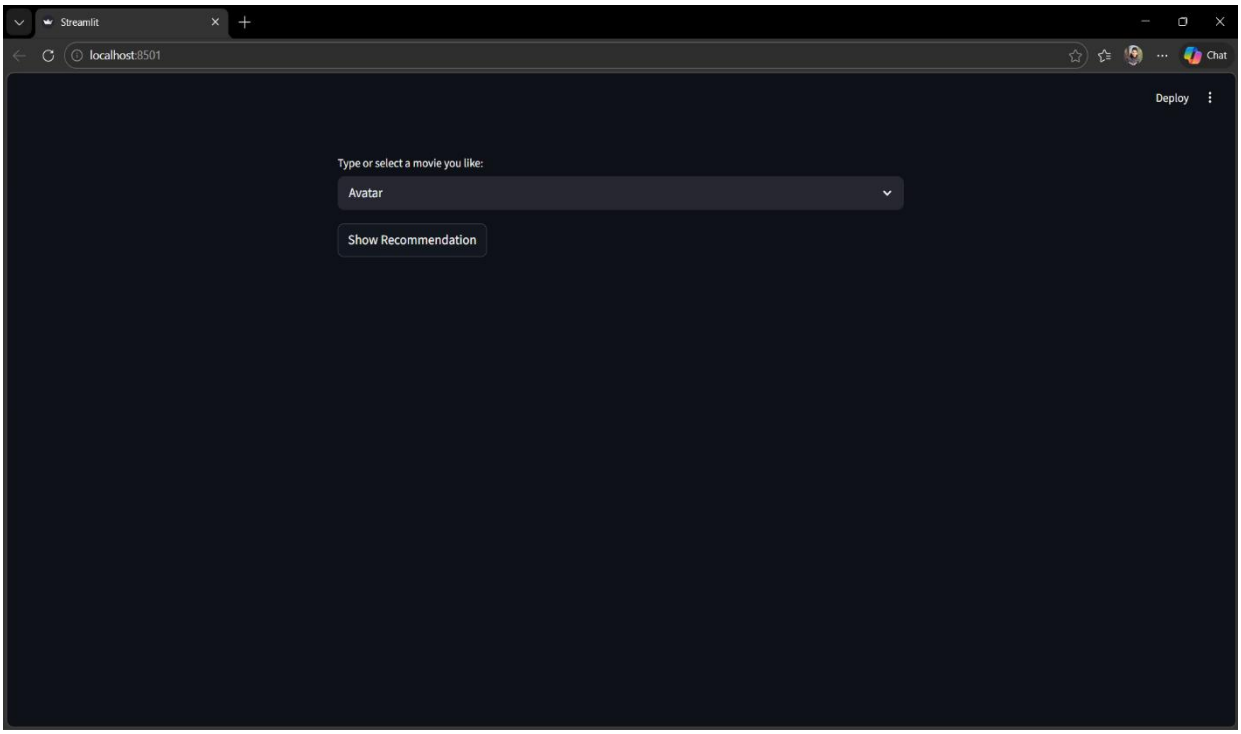
```
movie_list     = movie['title'].values
selected_movie = st.selectbox('Type or select a movie you like:', movie_list)


if st.button('Show Recommendation'):
    names, posters = recommender(selected_movie)
    col1, col2, col3, col4, col5 = st.columns(5)
    with col1: st.text(names[0]); st.image(posters[0])
    with col2: st.text(names[1]); st.image(posters[1])
    # ... repeated for col3, col4, col5
```

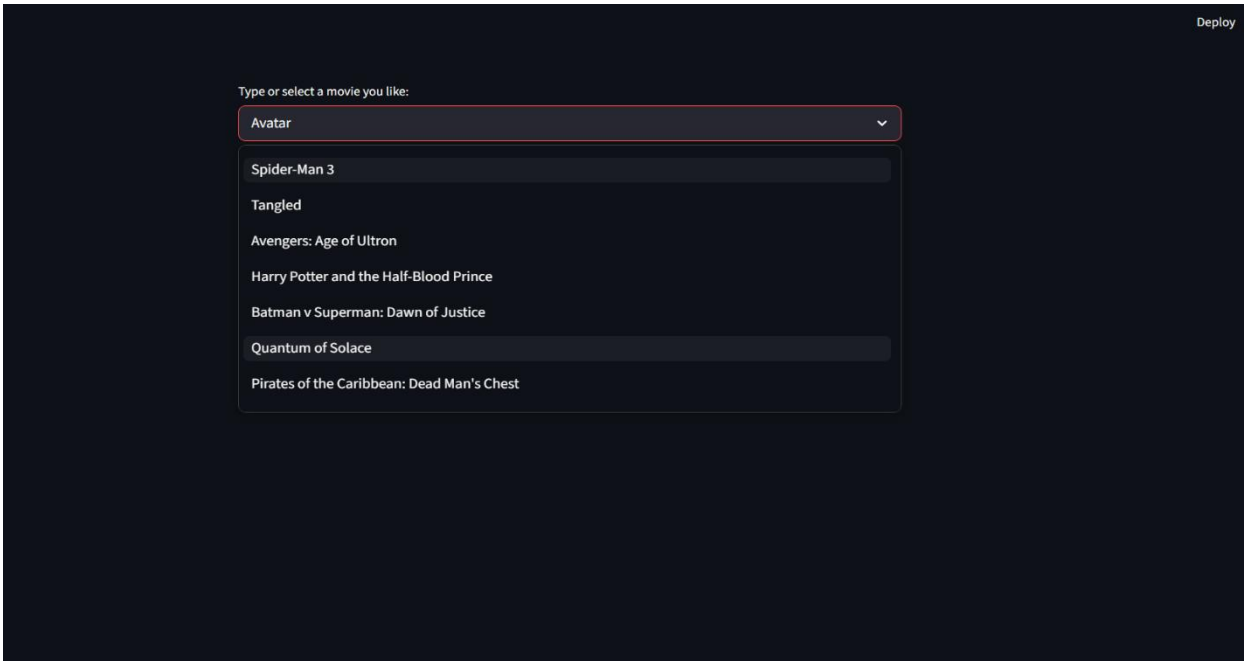
6.3 Deployment

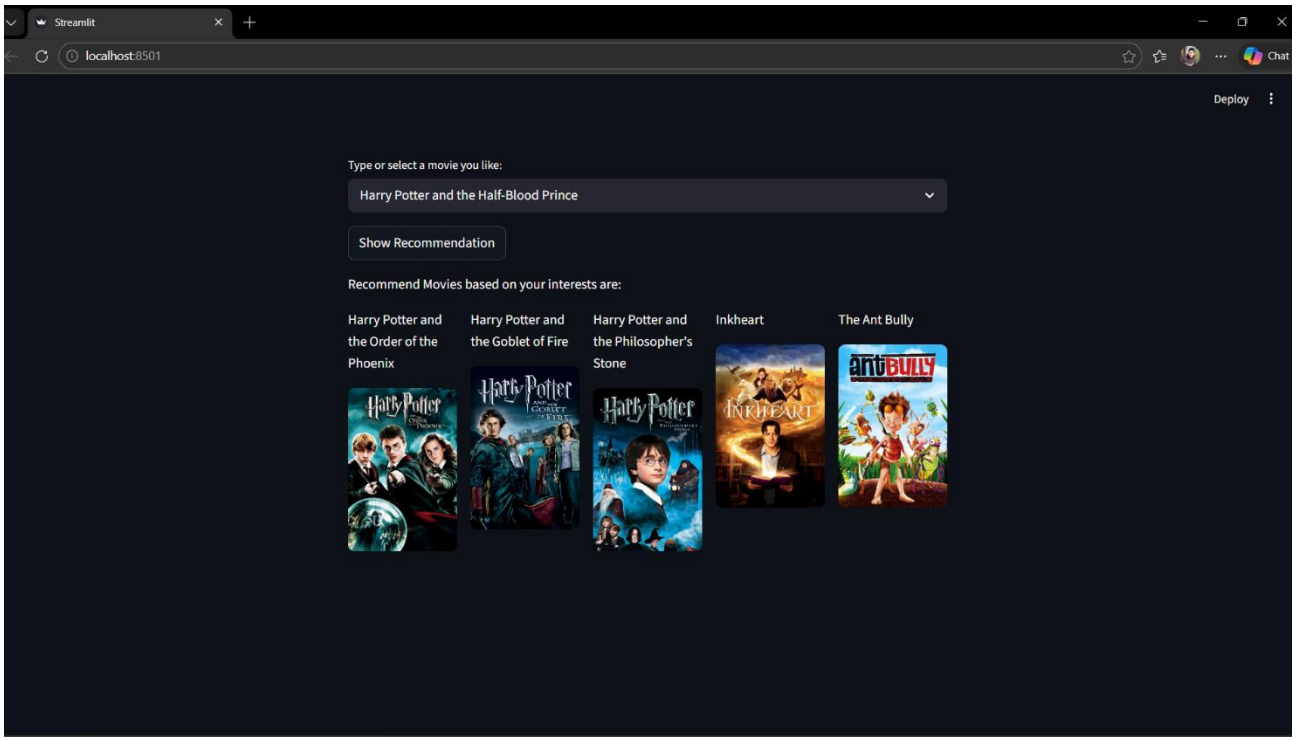
The application was deployed locally and accessed via the browser at http://localhost:8501. Streamlit's hot-reloading functionality enabled rapid iteration during development. No cloud deployment was configured for this phase of the project.

```
$ streamlit run app.py
```

```
Local URL:   http://localhost:8501
Network URL: http://192.168.x.x:8501
```







7. Tools and Libraries

Library / Tool	Version	Purpose in Project
Python	3.12.5	Core programming language for the entire pipeline
pandas	Latest	Data loading, merging, cleaning, and DataFrame operations
NumPy	Latest	Underlying numerical computation for vectorised operations
ast	Built-in	Safe parsing of stringified Python dict/list columns
NLTK	Latest	PorterStemmer for word stemming in NLP preprocessing
scikit-learn	Latest	CountVectorizer for text vectorisation; cosine_similarity for ML model
Streamlit	Latest	Web application framework for interactive localhost deployment
requests	Latest	HTTP GET calls to TMDB REST API for poster image URLs
pickle	Built-in	Serialisation and deserialisation of model artifacts (.pkl files)
Jupyter Notebook	Latest	Interactive development environment for training pipeline
VS Code	Latest	Primary IDE with Jupyter extension and Python virtual environment

8. Challenges and Solutions

8.1 JSON-Encoded String Columns

Challenge: The genres, keywords, cast, and crew columns were stored as strings representing Python list-of-dict structures, rather than native Python objects. A naive string split would not correctly extract the name values.

Solution: Python's ast.literal_eval() was used to safely evaluate each string as a Python literal. Custom functions were written to extract the relevant name or job fields from each parsed structure.

8.2 Multi-Word Token Collisions

Challenge: Feature values such as 'Science Fiction' or 'Sam Worthington' are multi-word phrases. If left as-is, CountVectorizer would treat each word individually, creating false co-occurrence signals (e.g., every movie with an 'action' genre would falsely match movies that simply use the word 'action' in their overview).

Solution: All spaces within multi-word tokens were removed before vectorisation, converting 'Science Fiction' to 'ScienceFiction'. This ensures each genre, keyword, or actor name is treated as a single indivisible token by the vectoriser.

8.3 Setting With Copy Warning

Challenge: Several .apply() operations triggered pandas' SettingWithCopyWarning because they were performed on a slice of the original DataFrame (new_df was created by selecting three columns from movie).

Solution: The warning is non-breaking and the operations produce correct results. A complete fix would require using .loc[] for all chained assignments (e.g., new_df.loc[:, 'tags'] = ...). This has been noted as a code-quality improvement for future versions.

8.4 Dataset Size Reduction

Challenge: The raw TMDB dataset contains 4,809 movies, but the final model operates on only 1,494. This is because the inner merge between the movies and credits files, combined with null value removal, eliminated rows with incomplete or mismatched data.

Solution: The reduced dataset still provides sufficient coverage for meaningful recommendations. Future work could investigate left joins, data imputation, or supplementary data sources to retain more movies.

9. Future Enhancements

9.1 Algorithm Improvements

- Collaborative Filtering: Integrate user-based or item-based collaborative filtering (SVD, ALS, matrix factorisation) using explicit user rating data for personalised recommendations.

- Hybrid Recommender System: Combine content-based and collaborative filtering for improved accuracy and cold-start handling.
- TF-IDF Vectorisation: Replace CountVectorizer with TF-IDF to down-weight common words and improve discrimination between movies.
- Word Embeddings: Use Word2Vec, GloVe, or BERT sentence embeddings for richer semantic understanding of movie content.

9.2 Data Enhancements

- Larger Dataset: Scale to the Full TMDB dataset (500,000+ entries) or integrate live TMDB API data for continuously updated recommendations.
- Additional Features: Incorporate user reviews, IMDb ratings, box office performance, and director filmography for richer feature representation.
- Multilingual Support: Extend to non-English language movies using multilingual NLP models.

9.3 Application Features

- User Rating System: Allow users to rate recommended movies and use that feedback to refine future recommendations.
- Advanced Filters: Add sidebar filters for genre, release year range, language, and minimum IMDb/TMDB rating.
- Search Autocomplete: Add a type-ahead search that ranks movies by relevance as the user types.
- Watch History: Implement session-based watch history to provide recommendations based on recently viewed movies.

9.4 Deployment

- Cloud Deployment: Deploy on Streamlit Community Cloud, Heroku, AWS EC2, or Google Cloud Run for public internet access.
- CI/CD Pipeline: Set up automated testing and deployment using GitHub Actions.
- Containerisation: Package the application in Docker for consistent, reproducible deployment across environments.
- API Layer: Expose the recommendation engine as a REST API (FastAPI or Flask) for integration into other applications.

10. Conclusion

This project successfully demonstrates the design, implementation, and deployment of a complete content-based movie recommendation system. Starting from raw TMDB CSV data, a full preprocessing pipeline was built to extract, clean, normalise, and engineer features from structured JSON-encoded columns. NLP techniques including tokenisation, space-normalisation, and Porter stemming were applied to create a unified ‘tags’ feature for each movie. Scikit-learn’s CountVectorizer transformed this text into a 1494 × 5000 feature matrix, and cosine similarity was computed over all movie pairs to produce a 1494 × 1494 similarity matrix.

The trained model and processed data were serialised using Pickle and integrated into a Streamlit web application that fetches live movie poster images from the TMDB API. Users can select any of the 1,494 available movies from a dropdown and receive five contextually similar recommendations, each displayed with its title and poster in an intuitive five-column layout on localhost.

The project achieves all its stated objectives and provides a strong foundation for future enhancements, including collaborative filtering, cloud deployment, and integration with larger movie databases. It demonstrates core competencies in data engineering, natural language processing, machine learning, and Python web application development.

11. References

1. TMDB 5000 Movie Dataset — Kaggle. Available at: https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata
2. The Movie Database (TMDB) API Documentation — https://developers.themoviedb.org/3
3. Scikit-learn Documentation — CountVectorizer and cosine_similarity. https://scikit-learn.org/stable/
4. NLTK Documentation — Porter Stemmer. https://www.nltk.org/
5. Streamlit Documentation — https://docs.streamlit.io/
6. pandas Documentation — https://pandas.pydata.org/docs/
7. Ricci, F., Rokach, L., & Shapira, B. (2015). Recommender Systems Handbook. Springer, New York.
8. Salton, G., & McGill, M. J. (1983). Introduction to Modern Information Retrieval. McGraw-Hill.